

Technical Design Document: Video Ingestion & Preprocessing Pipeline

PhD Application Technical Assessment

Candidate: Vishal Kumar

Date: 20 June 2026

1. Executive Summary

This document presents the architectural design and implementation of a self-contained video ingestion and preprocessing pipeline for sports computer vision applications. The system accepts raw team sports video footage and transforms it into structured, analysis-ready data suitable for downstream tasks such as player tracking, action recognition, and performance analytics.

The pipeline was designed with the following core principles:

- **Modularity** – Easy to extend and maintain
- **Configurability** – Behavior controlled via YAML files
- **Reproducibility** – Timestamped experiments with full configuration versioning
- **Performance** – Efficient processing with quality-aware filtering
- **Production Readiness** – Robust error handling and logging

2. Requirements Analysis

2.1 Functional Requirements

- Support multiple video container formats (MP4, MKV, AVI)
- Extract video metadata (duration, FPS, resolution, codec)
- Scene boundary detection and optional video splitting
- Configurable frame extraction with spatial transformations
- Per-frame quality assessment (blur detection)
- Human pose estimation using open-source models
- Structured JSON metadata manifest per run

2.2 Non-Functional Requirements

- Reasonable processing speed on consumer hardware
- Clear separation of concerns
- Comprehensive logging and experiment tracking
- Jupyter notebook for end-to-end demonstration

3. System Architecture

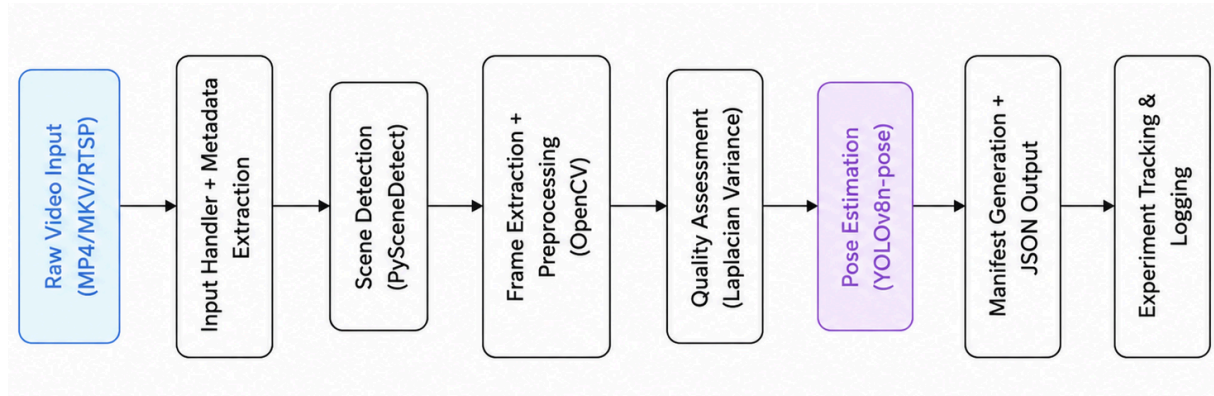


Fig 1. Data Flow

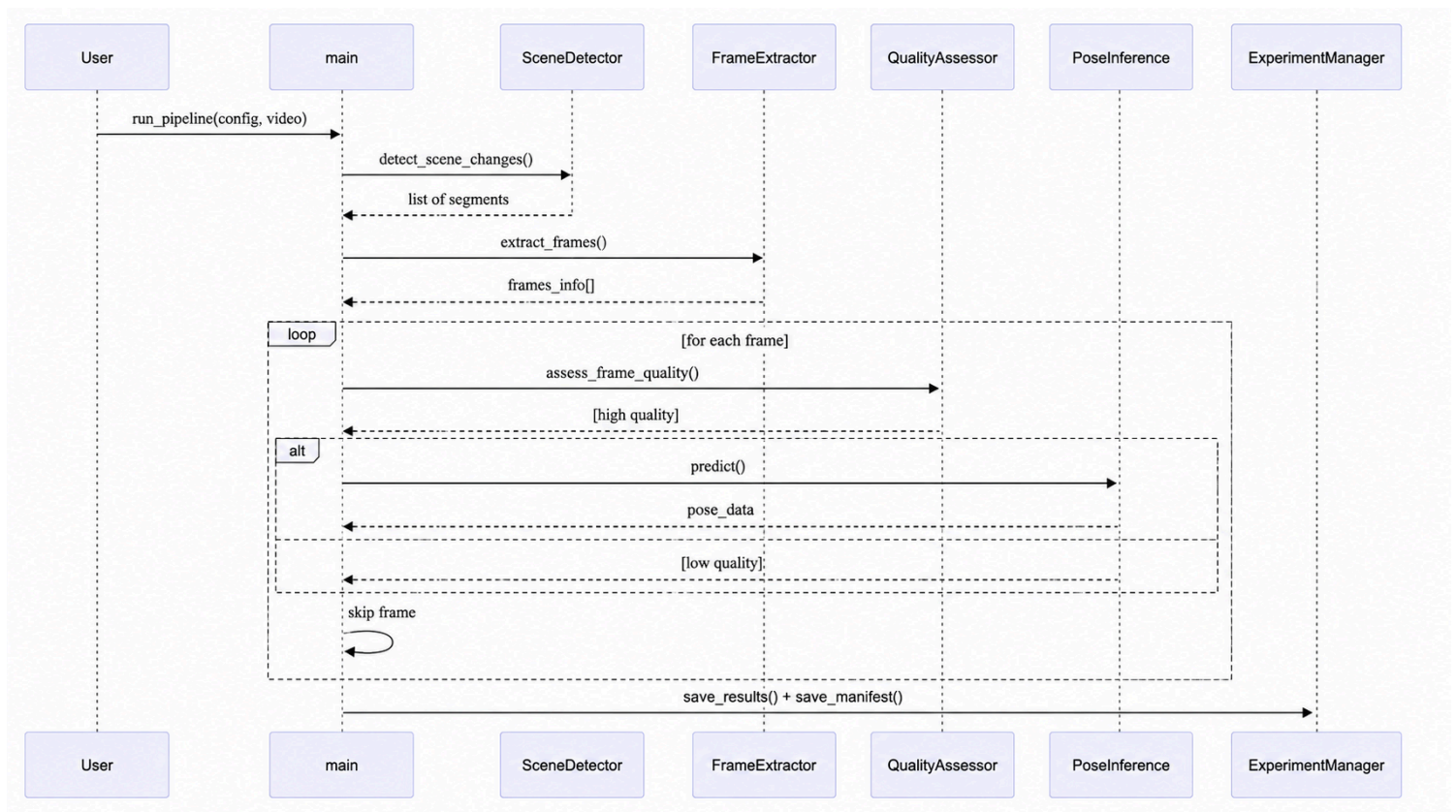


Fig 2. Sequence Diagram

4. Architectural Decisions & Trade-offs

4.1 Choice of Frameworks & Libraries

Component	Technology	Rationale	Trade-offs
Video I/O	OpenCV + ffmpeg-python	Mature, widely supported, excellent format compatibility	Higher memory usage vs pure ffmpeg
Scene Detection	PySceneDetect	Reliable content-based detection with good defaults	Slightly slower than custom optical flow
Pose Estimation	YOLOv8n-pose (Ultralytics)	State-of-the-art speed/accuracy balance, easy JSON export	Larger model size vs MediaPipe
Configuration	YAML	Human-readable, easy to version control	Less type safety than Pydantic
Experiment Tracking	Custom folder structure	Simple, no external database dependency	Less sophisticated than MLflow

4.2 Key Design Decisions

1. Modular Pipeline Design

The system follows a **pipeline pattern** with clear separation between core/ and utils/. This allows easy insertion of new processing stages (e.g., future optical flow or player detection).

2. Timestamped Output Directories

Every run creates a unique outputs/video_YYYYMMDD_HHMMSS/ folder. This ensures reproducibility and prevents overwriting previous results.

3. Quality-Aware Processing

Frames below a configurable quality threshold skip expensive pose inference. This provides significant performance gains on low-quality footage.

4. Hybrid Scene Detection

Fallback to single-segment processing when scene detection is disabled, ensuring robustness.

5. Detailed Component Design

5.1 Input Handling ()

- Argument parsing via argparse
- YAML configuration loading with CLI override support
- Robust path cleaning and validation

5.2 Video Utilities (utils/video_utils.py)

- `load_video_metadata()`: Uses `ffmpeg.probe()`
- `extract_frames()`: Decodes, resizes, converts color space
- `detect_scene_changes()`: Uses `ContentDetector` with optional video splitting

5.3 Core Pipeline (core/pipeline.py)

- Orchestrates quality assessment, segment assignment, and inference
- Progress tracking with `tqdm`

5.4 Pose Inference (core/inference.py)

- Loads YOLOv8n-pose model
- Returns rich JSON containing bounding boxes and keypoints

5.5 Experiment Management (core/experiment.py)

- Automatic timestamped experiment folders
- Saves config, metrics, and full results

6. Data Models & Output Format

Key Output: metadata.json

JSON

```
{  
  "frame_index": 0,  
  "timestamp": 0.0,  
  "path": "...",  
  "quality_score": 2074.45,  
  "segment": 0,  
  "pose_estimation": "[{ ... full YOLO detection ... }]"  
}
```

7. Scalability Considerations

7.1 Current Limitations

- Single-threaded processing
- All frames loaded into memory during pipeline run
- Local filesystem storage

7.2 Future Scalability Enhancements

- **Parallel Processing:** Use `concurrent.futures` or `Dask` for frame-level parallelism
- **Distributed Execution:** Containerization with `Docker` + `Kubernetes`
- **Streaming Support:** Add `RTSP/HLS` input for live match analysis

- **Cloud Integration:** Support for S3 input/output and serverless functions
- **GPU Optimization:** Leverage Ultralytics GPU acceleration
- **Batch Processing:** Process multiple videos in one experiment

8. Testing & Validation Strategy

- Unit tests for individual utilities
- Integration tests for full pipeline
- Manual validation using sample sports footage
- Performance benchmarking (FPS, memory usage)

9. Security & Reliability

- Graceful handling of corrupted videos
- Comprehensive logging
- Input validation and sanitization
- Deterministic output through fixed seeds where applicable

10. Conclusion & Future Work

This pipeline successfully meets all requirements of the PhD technical assessment while providing a solid foundation for a production sports intelligence platform.

Future Enhancements:

- Multi-camera synchronization
- Action recognition integration
- Real-time processing mode
- Advanced quality metrics (motion blur, lighting analysis)

The modular architecture ensures that new computer vision services (Amazon Rekognition, MediaPipe, etc.) can be easily integrated.